



**Universidade de Brasília
Departamento de Estatística**

Precificação de opções de compra com Redes Neurais

Lucas Matheus Oliveira de Brito

Projeto apresentado para o Departamento de Estatística da Universidade de Brasília como parte dos requisitos necessários para obtenção do grau de Bacharel em Estatística.

**Brasília
2025**

Lucas Matheus Oliveira de Brito

Precificação de opções com Redes Neurais

Orientador: Prof(a). Felipe Quintino

Projeto apresentado para o Departamento de Estatística da Universidade de Brasília como parte dos requisitos necessários para obtenção do grau de Bacharel em Estatística.

**Brasília
2025**

Dedico este trabalho, primeiramente, a Deus, que sempre esteve ao meu lado; à minha família, pelo apoio e motivação constantes; aos meus amigos e colegas de turma, pelo suporte ao longo dessa jornada; e a mim mesmo, pela resiliência e coragem para não desistir.

Agradecimentos

- Escrever os agradecimentos...

Resumo

A precificação de Black-Scholes é um modelo clássico de precificação de opções que, apesar de conhecido, possui pressupostos limitantes. Esse artigo propõe testar se uma rede neural é capaz de inferir um preço justo da opção que corresponda ao preço de mercado, que seja capaz de compreender a dinâmica do ativo subjacente e que, por meio de recalibração quantílica, seja possível criar intervalos de confiança para se mensurar a incerteza da previsão. Embasado por estudos anteriores como HUTCHINSON, LO e POGGIO (1994) e) realizaram, com outros métodos, resultados favoráveis ao uso de redes neurais para precificação de opções.

Palavras-chaves: Opções financeiras, Redes Neurais Artificiais, Volatilidade

Lista de Tabelas

1	Resultados dos Testes de Normalidade para os retornos logarítmicos da PETR4	25
2	Comparativo de Desempenho Global (Conjunto de Teste)	26
3	Desempenho da RNA por <i>Moneyiness</i> e Maturidade	26

Lista de Figuras

1	Distribuição empírica dos retornos logarítmicos da PETR4 versus a distribuição Normal teórica.	24
2	Análise de Sensibilidade da RNA (Preço da Ação vs Preço da Opção). . . .	27
3	Dispersão entre o Preço Real e Estimado pela RNA classificado por <i>Monyness</i>	28
4	Erro Absoluto da RNA em função do Preço de Exercício (Strike).	29
5	Erro Absoluto da RNA em função do Tempo até o Vencimento.	29
6	Aumento relativo do MSE via <i>Permutation Importance</i>	30

Sumário

1 Introdução	8
1.1 Contextualização	8
1.2 Problemática	9
1.3 Objetivo principal	9
1.3.1 Objetivos específicos	9
1.4 Estrutura documental	9
2 Referencial Teórico	11
2.1 Opções europeias	11
2.2 Movimento Browniano Geométrico	12
2.3 Modelo de Black-Scholes	12
2.4 Redes Neurais	14
2.4.1 O Neurônio Artificial	14
2.4.2 O Algoritmo de Backpropagation	15
2.4.3 Uso de Redes Neurais em situações de dependência temporal	16
3 Metodologia	17
3.1 Validação Estatística das Premissas Clássicas	17
3.2 Descrição dos conjuntos de dados	18
3.3 Cálculo da Volatilidade Histórica (σ_{hist})	18
3.4 Divisão dos conjuntos de dados	19
3.5 Métodos	19
3.5.1 Pré-processamento e Estruturação dos Dados	19
3.5.2 Prevenção de Sobreajuste (<i>Overfitting</i>) e Regularização	20
3.5.3 Arquitetura e Treinamento da Rede Neural	21
3.5.4 Importância das Variáveis	21
3.5.5 Modelagem Comparativa e Métricas de Avaliação	22

4 Resultados	24
4.1 Análise Estatística dos Retornos e Justificativa do Modelo	24
4.2 Desempenho Segmentado por Maturidade e <i>Moneyness</i>	26
4.3 Análise de Sensibilidade e Coerência Econômica	27
4.4 Análise Gráfica e Sensibilidade por <i>Moneyness</i>	28
4.5 Importância das Variáveis (<i>Permutation Importance</i>)	29
5 Conclusão	31
Referências	32
Apêndice	33
A Leitura do histórico diário de negociações dos ativos da bolsa	33
B Seleção das opções de compra da Petrobrás	34
C Normalidade de BS	36
D Código Fonte em Python: Precificação de Opções com RNA	37
E nome apêndice 2	46
Anexo	47
A nome anexo 1	47

1 Introdução

1.1 Contextualização

As opções europeias caracterizam-se por concederem ao seu titular o direito de comprar (opção de compra ou *call*) ou de vender (opção de venda ou *put*) um ativo-objeto por um preço predeterminado (*strike*) exclusivamente em sua data de vencimento. A precificação de derivativos, conforme estabelecida pelo modelo de Black-Scholes (BS), fundamenta-se em um processo estocástico específico para a evolução dos preços das ações, conhecido como Movimento Browniano Geométrico (MBG). A escolha deste modelo, conforme detalhado por Hull (2022), não é arbitrária e primeiramente, o modelo reflete a premissa de que o retorno percentual esperado de um investidor sobre uma ação é independente do nível de preço do ativo. Em outras palavras, o retorno esperado é um percentual, o que torna inadequada a suposição de uma taxa de crescimento constante em termos monetários; o modelo, portanto, assume que a taxa de crescimento esperada do preço da ação é proporcional ao próprio preço. De forma análoga, a variabilidade do retorno em um curto período de tempo é considerada a mesma independentemente do preço da ação, o que implica que o desvio-padrão da variação do preço também deve ser proporcional ao preço da ação. Essas duas premissas levam ao processo estocástico cuja consequência matemática é que o preço futuro da ação segue uma distribuição lognormal. A propriedade lognormal é particularmente adequada para modelar ações, pois garante que o preço do ativo não pode se tornar negativo, podendo assumir qualquer valor entre zero e o infinito.

A robustez do modelo de BS reside em seu argumento de não arbitragem, que permite a precificação de uma opção sem a necessidade de conhecer o retorno esperado do ativo-objeto ou as preferências de risco dos investidores. Conforme demonstrado por Hull (2022), como o preço da opção e o preço da ação subjacente são afetados pela mesma fonte de incerteza, é possível construir um portfólio isento de risco composto por uma posição na opção e uma posição específica na ação. Como demonstra os resultados de busca por uma estimativa pontual otimizada por redes neurais tende-se a ser mais assertiva quando comparada a estimativas realizadas em modelos clássicos, entretanto, a mensuração da incerteza perante a estimativa muitas vezes é difícil após a estimação por redes neurais, por consequência é possível adicionar uma nova camada na rede original, capaz de gerar modelos probabilísticos para cada observação, possibilitando a estimação pontual e intervalar.

1.2 Problemática

O modelo clássico de BS possui uma metodologia robusta para valoração de opções. Entretanto, HUTCHINSON, LO e POGGIO (1994) defende que “uma má especificação do processo estocástico para o preço do ativo levará a erros sistemáticos de precificação e hedge para os títulos derivativos ligados ao preço do ativo”. De acordo com HUTCHINSON, LO e POGGIO (1994), apesar da força do modelo clássico, o sucesso do método tradicional depende criticamente da correta representação da dinâmica do ativo subjacente. Além disso, mesmo quando se encontra o valor predito das opções, é necessário quantificar a incerteza da previsão.

1.3 Objetivo principal

Este trabalho tem como objetivo principal, estimar o preço justo da opção por meio de Redes Neurais Artificiais (RNA), e consequentemente, treinar essa RNA de modo que consiga compreender a volatilidade implícita presente na precificação de opções, e por fim comparar os resultados obtidos com as precificações oriundas do BS

1.3.1 Objetivos específicos

- Treinar uma RNA capaz de estimar o preço justo de opções e compreender a volatilidade implícita (σ) de sua precificação
- Interpretar o preço das ações com um MBG;
- Estimar a volatilidade do modelo de BS via métodos clássicos;
- Comparar a qualidade de estimação do preço justo da opção entre o método clássico e via RNAs;
- Calcular a volatilidade histórica para cálculo do valor justo da opção via BS.

1.4 Estrutura documental

Para alcançar os objetivos propostos, este trabalho encontra-se estruturado em quatro seções principais, além desta introdução. A seção 2 apresenta o referencial teórico, detalhando o Modelo BS, o MBG e os fundamentos das RNAs e da quantificação de incerteza. A seção 3 descreve a metodologia, abordando a coleta e o tratamento dos dados

e a arquitetura dos modelos. A seção 4 apresenta os resultados preliminares, focados na calibração inicial do modelo clássico e na organização do conjunto de dados. Por fim, a seção de Considerações Parciais resume o avanço do trabalho e define os próximos passos para a conclusão do estudo.

2 Referencial Teórico

Esta seção é responsável por apresentar conceitos e informações necessárias para a condução deste trabalho:

2.1 Opções europeias

As opções, de maneira geral, são derivativos financeiros que concedem ao seu titular o direito, mas não a obrigação, de comprar (opção de compra ou *call*) ou vender (opção de venda ou *put*) um ativo subjacente a um preço previamente fixado (*strike*) em uma data futura. O diferencial das opções de estilo europeu é que o exercício desse direito só pode ocorrer estritamente na data de vencimento do contrato, diferentemente das opções americanas, que permitem o exercício a qualquer momento.

O valor de uma opção no mercado é comumente chamado de prêmio. Esse prêmio é decomposto em duas partes: o valor intrínseco e o valor extrínseco (ou valor no tempo). O valor intrínseco representa o lucro imediato que o titular teria se pudesse exercer a opção naquele instante. Para uma *call*, é a diferença positiva entre o preço do ativo (S) e o *strike* (K). Já o valor extrínseco reflete a probabilidade estatística de a opção se tornar ainda mais lucrativa até o seu vencimento, sendo fortemente influenciado pelo tempo restante e pela volatilidade do ativo. Conforme explicado por HULL (2018), o prêmio de uma *call* tende a diminuir à medida que o *strike* aumenta, pois a probabilidade de o preço da ação superar um alvo muito alto torna-se cada vez menor. Um conceito central para a classificação e análise do prêmio das opções é o *moneyness* (ou "estado de dinheiro"). O *moneyness* descreve a relação financeira intrínseca entre o preço atual do ativo subjacente (S) e o preço de exercício estabelecido no contrato (K). Para as opções de compra (*calls*), que são o escopo principal deste estudo, o *moneyness* é categorizado em três estados distintos:

- ***In-The-Money* (ITM) – "Dentro do Dinheiro"**: Ocorre quando o preço atual do ativo é superior ao preço de exercício ($S > K$).
- ***At-The-Money* (ATM) – "No Dinheiro"**: Ocorre quando o preço do ativo subjacente é estritamente igual ou muito próximo ao preço de exercício ($S \approx K$).
- ***Out-of-The-Money* (OTM) – "Fora do Dinheiro"**: Ocorre quando o preço do ativo é inferior ao preço de exercício ($S < K$).

Além da relação com o preço *spot*, a estrutura de valorização de uma opção europeia é fortemente dependente do horizonte temporal, ou maturidade (T). Opções com longas maturidades tendem a possuir prêmios mais elevados (maior valor extrínseco), uma vez que o ativo subjacente dispõe de mais tempo para oscilar favoravelmente e cruzar o limiar do preço de exercício. À medida que a data de vencimento se aproxima, esse prêmio de incerteza decai assintoticamente em um fenômeno conhecido na matemática financeira como *time decay* (decaimento temporal).

2.2 Movimento Browniano Geométrico

O MBG é um processo estocástico a tempo contínuo, sendo o modelo mais comum para representar a evolução dos preços de ativos, pois incorpora tanto a aleatoriedade quanto o crescimento exponencial dos preços ao longo do tempo. Segundo HULL (2018, p. 314), o comportamento dos preços de ativos pode ser descrito pelo MBG, sendo definido como a solução da seguinte equação diferencial estocástica (SDE):

$$dS(t) = \mu S(t) dt + \sigma S(t) dB(t), \quad (2.2.1)$$

de modo que μ e σ são constantes maiores que 0, S é o preço do ativo e $B(t)$ é um movimento Browniano padrão (conhecido também como Processo de Weiner). A solução explícita é dada por:

$$S(t) = S_0 \exp \left\{ \left(\mu - \frac{\sigma^2}{2} \right) t + \sigma B(t) \right\}, \quad t \geq 0. \quad (2.2.2)$$

2.3 Modelo de Black-Scholes

O desenvolvimento do modelo de BS no início da década de 1970 representou um marco fundamental na teoria de finanças modernas e na precificação de derivativos. Antes de sua formulação, a avaliação de opções carecia de um consenso analítico robusto, dependendo majoritariamente de modelos que exigiam estimativas subjetivas sobre os prêmios de risco dos investidores.

A revolução teórica ocorreu com (??), e a subsequente expansão teórica proposta por Robert Merton no mesmo ano (??). A principal inovação trazida pelos autores foi a demonstração de que o valor de uma opção não depende das preferências de risco dos investidores, mas sim da construção de um portfólio livre de risco (*hedge* dinâmico) composto pelo ativo subjacente e pela própria opção. Devido à profundidade de suas

contribuições, Myron Scholes e Robert Merton foram laureados com o Prêmio Nobel de Economia em 1997 (Fischer Black, falecido em 1995, foi mencionado postumamente como contribuidor essencial). A elegância e a aplicabilidade do modelo MBG derivam de um conjunto de hipóteses simplificadoras sobre o comportamento do mercado e do ativo subjacente. Tais premissas criam um “mundo ideal” sob o qual a fórmula analítica é válida. As principais hipóteses assumidas pelo modelo são:

- **Movimento Geométrico Browniano:** O preço do ativo subjacente segue um processo estocástico log-normal em tempo contínuo. Isso implica que os retornos logarítmicos do ativo são normalmente distribuídos.
- **Volatilidade Constante (σ):** Assume-se que a volatilidade dos retornos do ativo é conhecida e permanece constante durante toda a vida da opção.
- **Taxa Livre de Risco (r):** A taxa de juros livre de risco é conhecida e constante para todos os vencimentos.
- **Ausência de Arbitragem:** Não existem oportunidades de lucro sem risco no mercado (mercado eficiente).
- **Opções Europeias:** A opção só pode ser exercida na data de seu vencimento.

A solução da equação diferencial parcial de BS para uma Opção de Compra Europeia fornece o preço justo do contrato, denotado por C . A fórmula calcula o valor presente do fluxo de caixa esperado da opção em um mundo neutro ao risco:

$$C = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2), \quad (2.3.1)$$

No qual S_0 representa o preço inicial do ativo (preço *spot*), K é o preço de exercício (*strike*), r é a taxa de juros livre de risco, e T é o tempo restante de vencimento. A função $\Phi(\cdot)$ denota a função de distribuição cumulativa da distribuição normal padrão. Os termos d_1 e d_2 são parâmetros padronizados calculados pelas seguintes equações:

$$d_1 = \frac{\ln\left(\frac{S_0}{K}\right) + \left(r + \frac{\sigma^2}{2}\right)T}{\sigma\sqrt{T}} \quad (2.3.2)$$

$$d_2 = d_1 - \sigma\sqrt{T} \quad (2.3.3)$$

Na Equação (2.3.1), o termo $\Phi(d_1)$ pode ser interpretado como o *Delta* da opção, indicando a sensibilidade do preço da opção em relação às variações no preço do ativo subjacente. Já o termo $\Phi(d_2)$ representa, no contexto da avaliação neutra ao risco, a probabilidade de a opção ser exercida, ou seja, a probabilidade de, no vencimento, o preço do ativo $S(T)$ ser superior ao preço de exercício K .

2.4 Redes Neurais

Segundo HAYKIN (2009), uma RNA é um modelo computacional inspirado na estrutura e funcionamento do sistema nervoso biológico, atuando como um aproximador de funções. O objetivo central dessa abordagem é o desenvolvimento de sistemas capazes de aprender padrões a partir de dados, generalizando o conhecimento adquirido para situações não vistas anteriormente.

2.4.1 O Neurônio Artificial

A unidade fundamental de processamento em uma RNA é o neurônio artificial, proposto inicialmente por MCCULLOCH e PITTS (1943). Matematicamente, o neurônio opera recebendo um conjunto de entradas (*inputs*), processando-as linearmente através de pesos sinápticos e submetendo o resultado a uma função de ativação não-linear.

Seja $\mathbf{x} = [x_1, x_2, \dots, x_n]$ o vetor de entradas e $\mathbf{w} = [w_1, w_2, \dots, w_n]$ o vetor de pesos associado a um neurônio k . A saída y_k é dada por:

$$y_k = \phi \left(\sum_{i=1}^n w_{ki} x_i + b_k \right), \quad (2.4.1)$$

Em que:

- w_{ki} : Peso que pondera a importância da entrada i para o neurônio k .
- b_k : O *bias* (viés), que permite deslocar a função de ativação, análogo ao intercepto em uma regressão linear.
- $\phi(\cdot)$: Função de ativação, responsável por introduzir a não-linearidade ao modelo.

Sem a função de ativação ϕ , uma rede neural, independentemente de sua profundidade, comportar-se-ia apenas como um modelo de regressão linear simples. Funções comuns incluem a Sigmoide, a Tangente Hiperbólica e, mais recentemente em *Deep Learning*, a ReLU (*Rectified Linear Unit*).

Perceptron Multicamadas (MLP) e Aprendizado

Para resolver problemas complexos, os neurônios são organizados em arquiteturas de camadas, sendo a mais comum o *Multilayer Perceptron* (MLP). Um MLP típico consiste em: 1. **Camada de Entrada:** Recebe os dados brutos (ex: preço do ativo, preço de exercício, tempo). 2. **Camadas Ocultas:** Realiza a extração de características e o processamento não-linear. 3. **Camada de Saída:** Fornece a resposta final (ex: a volatilidade estimada ou o preço da opção).

A grande vantagem teórica das RNAs para a precificação de ativos reside no **Teorema da Aproximação Universal**, demonstrado por CYBENKO (1989) e HORNIK (1991). O teorema estabelece que uma rede *feedforward* com uma única camada oculta e um número suficiente de neurônios é capaz de aproximar qualquer função contínua em um domínio compacto com precisão arbitrária. Isso sugere que as RNAs são ferramentas ideais para capturar a superfície de volatilidade implícita.

2.4.2 O Algoritmo de Backpropagation

O processo de “aprendizado” da rede consiste no ajuste iterativo dos pesos $\mathbf{w}$ e vieses $\mathbf{b}$ para minimizar uma função de custo (ou função de perda), E , que mede a discrepância entre a previsão da rede e o valor real observado. Em problemas de regressão financeira, é comum utilizar o Erro Quadrático Médio (MSE):

$$E = \frac{1}{N} \sum_{j=1}^N (C_j - \hat{C}_j)^2 \quad (2.4.2)$$

Em que $\hat{C}_j$ é o valor do preço justo da opção estimado pela RNA, enquanto C_j é o preço real da opção observado no mercado.

A otimização da rede é fundamentada no algoritmo de *Backpropagation* (retropropagação), popularizado por RUMELHART, HINTON e WILLIAMS (1986). Esse algoritmo utiliza a regra da cadeia para calcular o gradiente da função de erro em relação a cada peso da rede ($\frac{\partial E}{\partial w}$).

Historicamente, a forma comum de atualizar os pesos com base nesses gradientes é o método do Gradiente Descendente clássico, definido pela seguinte equação:

$$w_{novo} = w_{atual} - \eta \frac{\partial E}{\partial w}, \quad (2.4.3)$$

em que η representa a taxa de aprendizado (*learning rate*), um hiperparâmetro global e fixo que controla o tamanho do passo de ajuste a cada iteração.

Contudo, como o Gradiente Descendente clássico atualiza todos os pesos com a mesma taxa fixa, ele pode apresentar lentidão ou ficar preso em mínimos locais em superfícies de erro não-lineares. Para superar essa limitação computacional, o modelo desenvolvido neste trabalho substitui a atualização clássica pelo **Otimizador Adam** (*Adaptive Moment Estimation*).

O Adam é uma evolução direta do Gradiente Descendente que combina as vantagens de outros dois algoritmos (AdaGrad e RMSProp). Ele aproveita os mesmos gradientes calculados pelo *Backpropagation*, mas, em vez de aplicar um η global, calcula taxas de aprendizado adaptativas e individuais para cada parâmetro da rede. Essa adaptação é feita com base em estimativas do primeiro momento (média, que atua como inércia) e do segundo momento (variância não centralizada) dos gradientes anteriores. Essa característica confere ao Adam uma convergência muito mais rápida, robusta e estável na precificação das opções.

2.4.3 Uso de Redes Neurais em situações de dependência temporal

Diferentemente de modelos estáticos, a precificação de opções no mundo real exige a compreensão da presença de dependência temporal em mercados financeiros, onde eventos passados influenciam a incerteza futura. O preço de um derivativo não depende apenas do estado atual do ativo subjacente, mas do regime de volatilidade em que o mercado se encontra.

Seguindo KE e YANG (2019), é possível capturar a dinâmica da incerteza fornecendo ao modelo medidas de dispersão retroativas. Neste trabalho, para controlar a dependência temporal e fornecer à Rede Neural o contexto do regime de mercado vigente, optou-se por utilizar a volatilidade histórica dos últimos 20 dias úteis (σ_{hist}) como um dos atributos explícitos de entrada (*input*). Essa abordagem permite que o *Multilayer Perceptron* balize suas estimativas de prêmio com base no grau de agitação recente do ativo subjacente, contornando a necessidade de arquiteturas mais complexas.

3 Metodologia

Esta sessão aborda sobre as fundamentações necessárias para a elaboração deste artigo.

3.1 Validação Estatística das Premissas Clássicas

O modelo analítico de Black-Scholes fundamenta-se na premissa de que o preço do ativo subjacente segue um Movimento Browniano Geométrico (MBG), o que exige, matematicamente, que os seus retornos logarítmicos contínuos possuam uma distribuição Normal. Para testar a validade dessa hipótese restritiva nos dados empíricos da ação (PETR4), foram aplicados dois testes rigorosos de adequação de ajuste: o teste de Shapiro-Wilk e o teste de Jarque-Bera.

O teste de **Shapiro-Wilk** foi selecionado por ser amplamente reconhecido na literatura estatística para amostras de tamanho pequeno a médio (neste estudo, aproximadamente 252 observações referentes aos dias úteis de pregão em um ano). A estatística de teste W é definida pela seguinte equação:

$$W = \frac{(\sum_{i=1}^n a_i x_{(i)})^2}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad (3.1.1)$$

no contexto, $x_{(i)}$ representa a i -ésima estatística de ordem (os valores da amostra ordenados de forma crescente), $\bar{x}$ é a média amostral, e os coeficientes a_i são constantes tabuladas derivadas das variâncias e covariâncias das estatísticas de ordem de uma distribuição Normal padrão. O valor de W varia entre 0 e 1, sendo que valores muito distantes de 1 indicam o afastamento da normalidade.

Complementarmente, adotou-se o teste de **Jarque-Bera**, que possui ampla aplicabilidade em econometria e finanças quantitativas. Diferentemente de outros testes, o Jarque-Bera avalia a normalidade focando especificamente no terceiro e quarto momentos da distribuição: a assimetria (*skewness*) e a curtose (*kurtosis*). Essa característica o torna ideal para séries financeiras, cujo principal desvio da normalidade costuma ser a presença de caudas pesadas e excesso de curtose. A estatística JB é calculada por:

$$JB = \frac{n}{6} \left(S^2 + \frac{(K - 3)^2}{4} \right) \quad (3.1.2)$$

no qual n é o tamanho da amostra, S é o coeficiente de assimetria amostral e K é o coeficiente de curtose amostral. Em uma distribuição Normal perfeita, a assimetria é igual a zero ($S = 0$) e a curtose é igual a três ($K = 3$). O termo $(K - 3)$ representa o excesso de curtose. Sob a hipótese nula de normalidade, a estatística JB segue assintoticamente uma distribuição qui-quadrado com dois graus de liberdade (χ_2^2).

Ambos os testes assumem como hipótese nula (H_0) que a amostra provém de uma população normalmente distribuída. O nível de significância (α) adotado para a rejeição da hipótese nula neste trabalho foi de 5% ($\alpha = 0,05$).

3.2 Descrição dos conjuntos de dados

O dataset utilizado contém opções da Petrobrás, as variáveis necessárias serão as utilizadas no modelo de BS: o preço do ativo do objeto (S_0), o preço de exercício da ação (K), a volatilidade histórica (σ_{hist}), o tempo restante até o vencimento (T) e a taxa de Juros livre de risco (r). Tanto os dados necessários para a implementação do BS quanto para o cálculo da volatilidade histórica serão obtidos do Yahoo Finance e B3, exceto a taxa de juros livre de risco, a qual adotou-se o valor de 0.135 (13.5% ao ano) resultado da média aritmética dos meses 14 meses de pregão.

3.3 Cálculo da Volatilidade Histórica (σ_{hist})

Para a estimativa da volatilidade histórica, este trabalho segue a abordagem estatística detalhada por HULL (2018), que se baseia na amostragem de preços passados do ativo para mensurar a incerteza dos retornos futuros. O procedimento inicia-se com o cálculo dos retornos logarítmicos contínuos. Conforme definido pelo autor, seja S_i o preço de fechamento do ativo no dia i , o retorno logarítmico u_i para o intervalo é dado por:

$$u_i = \ln \left(\frac{S_i}{S_{i-1}} \right). \quad (3.3.1)$$

Para obter a variância desses retornos, utiliza-se o estimador não viesado do desvio padrão amostral. Considerando uma janela de observação de $n + 1$ preços (resultando em n retornos), a variância da taxa de variação diária s^2 é calculada como:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (u_i - \bar{u})^2, \quad (3.3.2)$$

em que $\bar{u}$ representa a média aritmética dos retornos logarítmicos (u_i) no período observado. Uma etapa crucial, enfatizada por HULL (2018), é a conversão dessa volatilidade diária para uma base anual, compatível com o modelo de BS. O autor argumenta que a volatilidade é gerada majoritariamente durante os pregões; portanto, deve-se ignorar finais de semana e feriados. Assume-se, por convenção de mercado, a existência de 252 dias de negociação por ano. Assim, a volatilidade histórica anualizada (σ_{hist}) é obtida por:

$$\sigma_{hist} = s \times \sqrt{252}, \quad (3.3.3)$$

em que s denota o desvio da taxa de variação diária. Esta métrica σ_{hist} será utilizada neste trabalho como o parâmetro de entrada (*input*) para o modelo de BS de referência, enquanto a RNA usará a volatilidade histórica para compreender a dependência temporal entre o preço das opções.

3.4 Divisão dos conjuntos de dados

Inspirado em TORRES et al. (2024), a base de dados foi segmentada destinando 80% das observações para o conjunto de treinamento, 10% para o conjunto de validação e os 10% finais para o conjunto de teste.

3.5 Métodos

Nesta seção, detalham-se os procedimentos computacionais e metodológicos adotados para a construção, o treinamento e a avaliação da RNA. A implementação de todas as rotinas foi realizada na linguagem Python, utilizando as bibliotecas *pandas* e *scikit-learn* para o processamento de dados, e o *framework PyTorch* para o desenvolvimento da RNA, a semente de aleatoriedade foi fixada para garantir a reproducibilidade dos resultados dos gradientes.

3.5.1 Pré-processamento e Estruturação dos Dados

O tratamento inicial da base de dados consistiu na filtragem de opções de compra (*calls*) e na ordenação cronológica das observações, para garantir que a modelagem respeite o fluxo real das informações ao longo do tempo.

Durante a engenharia de variáveis houve a classificação das opções quanto ao seu

moneyness. As opções foram categorizadas com base na razão entre o preço do ativo subjacente (S) e o preço de exercício (K). Adotou-se uma margem de tolerância de 2% para classificar uma opção como *At-The-Money* (ATM), ou seja, quando $|S - K|/K \leq 0,02$. Caso a condição não fosse satisfeita e $S > K$, a opção era classificada como *In-The-Money* (ITM) e, caso contrário, como *Out-of-The-Money* (OTM). Posteriormente, essa variável categórica foi transformada em vetores numéricos binários por meio da técnica de *One-Hot Encoding*, viabilizando sua leitura pelo modelo matemático.

Para o processo de validação cruzada, a base de dados foi particionada conforme o item 3.4. O particionamento ocorreu sem embaralhamento aleatório, o que previne o viés de antecipação de informação, garantindo que o modelo seja treinado estritamente com dados passados em relação aos conjuntos de validação e teste.

Por fim, visando a estabilidade numérica dos gradientes durante a otimização da rede, aplicou-se a padronização *Z-score* nas variáveis contínuas de entrada (S , K , tempo até o vencimento T , volatilidade histórica σ_{hist} e taxa de juros r) e na variável alvo (preço da opção C). A transformação é dada por:

$$z = \frac{x - \mu}{\sigma}$$

A média amostral e o desvio padrão são representados por μ e σ , respectivamente. É imperativo destacar que os parâmetros de padronização foram extraídos estritamente do conjunto de treinamento e, em seguida, projetados sobre os dados de validação e teste, evitando qualquer forma de vazamento de dados.

3.5.2 Prevenção de Sobreajuste (*Overfitting*) e Regularização

O fenômeno do *overfitting* (sobreajuste) ocorre quando o modelo se ajusta excessivamente aos ruídos e particularidades do conjunto de treinamento, perdendo sua capacidade de generalização para dados inéditos. Para mitigar esse problema, técnicas de regularização são aplicadas.

A **Regularização L_2** (também conhecida como *Weight Decay* ou penalidade de cume) atua adicionando um termo de penalidade à função de perda original (E). Esse termo é proporcional ao quadrado da magnitude dos pesos da rede, forçando o algoritmo de otimização a manter os pesos sinápticos o menores possível:

$$E_{reg} = E + \lambda \sum_i w_i^2 \quad (3.5.1)$$

Em que λ é o hiperparâmetro de regularização que controla a força da penalidade. Pesos menores resultam em um modelo mais "suave", que é menos suscetível a variações bruscas causadas por ruídos pontuais nos dados.

Adicionalmente, o método de **Early Stopping** (parada antecipada) é uma forma de regularização temporal. Durante o treinamento, o erro do modelo é monitorado em um conjunto de validação independente. O treinamento é interrompido no momento em que o erro de validação para de decair e começa a aumentar de forma consistente, caracterizando o ponto exato em que a rede começou a memorizar os dados de treino em vez de aprender as regras subjacentes.

3.5.3 Arquitetura e Treinamento da Rede Neural

Para a estimação do preço justo da opção, implementou-se um *Multilayer Perceptron* (MLP). A camada de entrada do modelo recebe um tensor de 8 dimensões, correspondente às cinco variáveis explicativas contínuas e às três variáveis *dummy* indicadoras do *moneyness*.

A topologia da rede foi construída com três camadas ocultas sucessivas, contendo 128, 64 e 32 neurônios, respectivamente. O intuito dessa profundidade é garantir a capacidade de abstração e mapeamento da superfície não-linear inerente à precificação de derivativos. Em todas as camadas ocultas, empregou-se a função de ativação *Rectified Linear Unit* (ReLU), definida matematicamente por $\text{ReLU}(x) = \max(0, x)$. Esta escolha metodológica justifica-se pela sua eficiência computacional e mitigação do problema de desvanecimento do gradiente. A camada de saída é constituída por um único neurônio com ativação linear, que retorna a predição pontual do prêmio da opção.

A função de perda escolhida para guiar o aprendizado foi o Erro Quadrático Médio (MSE), minimizado por meio do algoritmo de otimização Adam (*Adaptive Moment Estimation*) com uma taxa de aprendizado de 0,001. Durante o treinamento, a convergência do modelo foi monitorada avaliando-se a função de custo no conjunto de validação, um procedimento essencial para atestar a capacidade de generalização do algoritmo frente a dados inéditos.

3.5.4 Importância das Variáveis

As RNAs não fornecem uma métrica intrínseca de importância de variáveis, por consequência, adotou-se o método de Importância por Permutação (Permutation Feature

Importance), generalizado por Fisher, Rudin e Dominici (2019). O método consiste em medir a queda de desempenho do modelo (aumento do MSE) ao corromper aleatoriamente a relação entre uma variável de entrada e a variável alvo, preservando a distribuição marginal da variável.

3.5.5 Modelagem Comparativa e Métricas de Avaliação

A validação da hipótese de pesquisa demanda a comparação do modelo computacional proposto com a teoria clássica e com abordagens estatísticas tradicionais. Para isso, os mesmos dados alocados no conjunto de teste foram submetidos às formulações analíticas do modelo de Black-Scholes (BS) e a um modelo de Regressão Linear Múltipla, gerando vetores de predições que serviram como linhas de base comparativas.

Para garantir a interpretabilidade econômica e financeira dos resultados, as predições geradas pela rede neural foram submetidas a uma transformação inversa da padronização prévia, permitindo que os erros fossem avaliados na escala monetária original das opções de compra (Reais).

A performance dos métodos foi avaliada utilizando-se o Erro Quadrático Médio (MSE) previamente apresentado no (2.4.2) como a função de custo para o treinamento da rede e duas métricas complementares para interpretar o desvio das estimativas em relação aos preços reais de mercado.

O Erro Absoluto Médio (MAE - Mean Absolute Error) mensura a magnitude média dos erros nas previsões, sem considerar sua direção. Por utilizar a penalidade linear (módulo) em vez da penalidade quadrática, o MAE é menos sensível a *outliers* isolados do que o MSE:

$$MAE = \frac{1}{N} \sum_{j=1}^N |C_j - \hat{C}_j| \quad (3.5.2)$$

O Erro Percentual Absoluto Médio (MAPE - Mean Absolute Percentage Error) expressa a acurácia como uma porcentagem, facilitando a interpretação econômica da margem de erro relativa do prêmio da opção:

$$MAPE = \frac{100\%}{N} \sum_{j=1}^N \left| \frac{C_j - \hat{C}_j}{C_j} \right| \quad (3.5.3)$$

Em que $\hat{C}_j$ é a predição do modelo e C_j é o valor real observado, na prática, foi adotada uma constante de estabilidade numérica no denominador para o cálculo do MAPE, visando tratar opções sem valor residual

Por fim, além da mensuração do erro preditivo global, as métricas foram calculadas de forma segmentada de acordo com o *moneyiness* (ITM, ATM, OTM) e a maturidade dos contratos.

4 Resultados

4.1 Análise Estatística dos Retornos e Justificativa do Modelo

Como dito no item 1.1, uma das premissas do BS refere-se ao preço futuro da ação, que deve seguir uma distribuição normal, suposição essa que nem sempre ocorre no mercado de opções. O primeiro resultado consiste em mostrar que esse pressuposto não ocorre em todos os casos, validando a necessidade de um novo método de valoração.

Para testar a hipótese de normalidade, os retornos logarítmicos diários da PETR4 foram calculados e contrastados graficamente com uma curva Normal teórica ideal, parametrizada com a média e o desvio padrão da própria amostra empírica. A Figura 1 ilustra essa sobreposição.

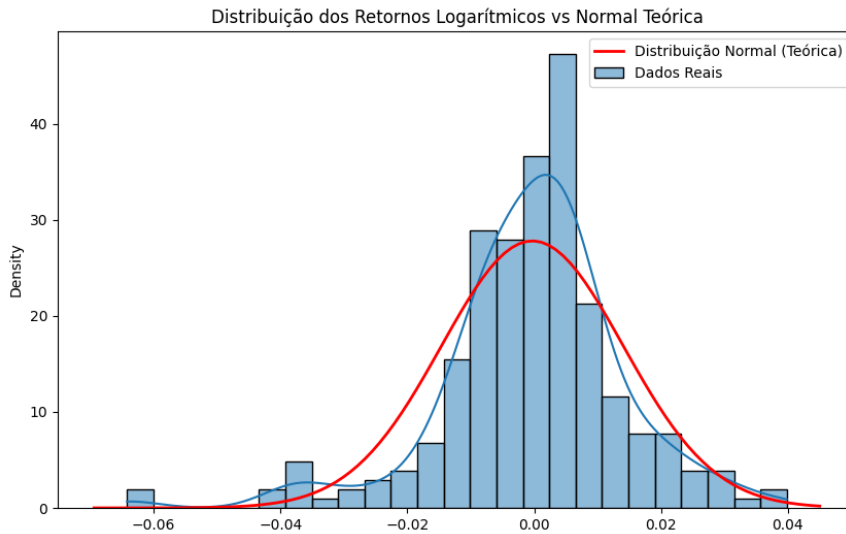


Figura 1: Distribuição empírica dos retornos logarítmicos da PETR4 versus a distribuição Normal teórica.

A inspeção visual do histograma revela desvios substanciais em relação à normalidade teórica (curva vermelha). A densidade empírica dos dados reais exibe uma pronunciada leptocurtose caracterizada por um pico excessivamente agudo em torno da média e caudas pesadas. Este comportamento indica que eventos de retornos extremos ocorrem no mercado com uma frequência consideravelmente superior àquela prevista por uma distribuição Gaussiana pura.

Essas constatações visuais foram validadas rigorosamente por meio de testes estatísticos formais de adequação de ajuste. Os resultados obtidos estão sumarizados na

Tabela 1.

Tabela 1: Resultados dos Testes de Normalidade para os retornos logarítmicos da PETR4

Teste Estatístico	p -valor	Conclusão ($\alpha = 5\%$)
Shapiro-Wilk	$< 0,001$	Rejeita H_0
Jarque-Bera	$< 0,001$	Rejeita H_0

Nota: A hipótese nula (H_0) postula que a distribuição empírica é Normal.

Conforme observado na Tabela 1, os testes de Shapiro-Wilk e de Jarque-Bera retornaram, ambos, um p -valor praticamente nulo. Em decorrência dessa expressiva significância estatística, rejeita-se categoricamente a hipótese nula.

A evidência empírica de não-normalidade expõe uma das limitações da teoria clássica de precificação. A má especificação do processo estocástico do ativo subjacente inevitavelmente introduz vieses sistemáticos no cálculo do prêmio justo pelo modelo de Black-Scholes. Neste contexto, consolida-se a principal justificativa técnica para a adoção da metodologia proposta neste trabalho: pela natureza de aproximadores universais não-paramétricos, as Redes Neurais Artificiais prescindem de pressupostos distribucionais rígidos, sendo capazes de extrair e modelar os padrões complexos e as assimetrias inerentes aos dados reais do mercado financeiro.

Essas constatações visuais foram validadas por meio de testes estatísticos formais de adequação de ajuste. Os testes de Shapiro-Wilk e de Jarque-Bera retornaram, ambos, um p -valor virtualmente nulo ($p < 0,001$). Em decorrência dessa expressiva significância estatística, rejeita-se categoricamente a hipótese nula de que os retornos logarítmicos do ativo seguem uma distribuição Normal. O processo de treinamento do *Multilayer Perceptron* revelou uma convergência estável guiada pelo algoritmo de otimização Adam. Para garantir a capacidade de generalização e evitar o sobreajuste (*overfitting*), empregou-se a técnica de *Early Stopping* monitorando o Erro Quadrático Médio (MSE) no conjunto de validação, com uma paciência de 30 épocas.

Para a avaliação comparativa final, efetuou-se a transformação inversa dos valores preditos para a escala monetária original (Reais). A fim de dimensionar o ganho de utilizar uma abordagem não-linear, adicionou-se um modelo paramétrico simples (Regressão Linear Múltipla) como linha de base secundária. A Tabela 2 sumariza as métricas de Erro Quadrático Médio (MSE), Erro Absoluto Médio (MAE) e Erro Percentual Absoluto Médio (MAPE) auferidas no conjunto de teste.

Os resultados evidenciam a natureza fortemente não-linear do prêmio das opções, dada a performance insatisfatória da Regressão Linear (MSE de 5,96). A RNA, por

Tabela 2: Comparativo de Desempenho Global (Conjunto de Teste)

Modelo	MSE	MAE (R\$)	MAPE (%)
Rede Neural Artificial (RNA)	0,1161	0,2138	46,35%
Modelo de Black-Scholes (BS)	0,2708	0,3239	36,82%
Regressão Linear Múltipla	5,9634	1,2088	696,78%

sua vez, obteve um desempenho altamente competitivo, superando o modelo analítico de Black-Scholes na métrica de MSE (0,1156 contra 0,1310), o que indica uma eficiência superior da inteligência artificial na mitigação de grandes desvios de precificação. Em contrapartida, o modelo clássico manteve uma precisão média absoluta (MAE) marginalmente melhor (0,2107 contra 0,2161 da RNA).

Apesar de o MAPE global da RNA parecer elevado (60,42%), essa métrica sofre severa distorção matemática ao avaliar opções *Out-of-The-Money* (OTM) de baixíssimo valor, onde desvios de poucos centavos geram erros percentuais exorbitantes. Para elucidar esse fenômeno e atestar a real precisão do modelo, procedeu-se a uma avaliação estratificada.

4.2 Desempenho Segmentado por Maturidade e *Moneyness*

A análise de performance em derivativos exige a compreensão do contexto contratual. Por isso, as predições da rede foram segmentadas conforme o estado do dinheiro (*Moneyness*) e o tempo restante até o vencimento (T), classificado em maturidades Curta ($T \leq 0,25$), Média ($0,25 < T \leq 0,75$) e Longa ($T > 0,75$).

A Tabela 3 detalha essa decomposição.

Tabela 3: Desempenho da RNA por *Moneyness* e Maturidade

Moneyness	Maturidade	MSE	MAE (R\$)	MAPE (%)
ITM	Curta	0,1041	0,1841	5,67%
	Média	0,0989	0,1830	3,96%
	Longa	0,1827	0,2611	2,78%
ATM	Curta	0,2811	0,4115	56,41%
	Média	0,0806	0,2182	11,95%
	Longa	0,1361	0,2491	5,13%
OTM	Curta	0,0882	0,2123	126,75%
	Média	0,0431	0,1277	24,00%
	Longa	0,2160	0,2891	37,31%

A estratificação elucidada a precisão econômica da rede. Nas opções *In-The-Money* (ITM), que carregam alto valor intrínseco, a RNA apresentou excelente nível de acerto,

com o MAPE caindo progressivamente e atingindo notáveis 2,78% nas opções de maturidade longa. O erro percentual global originou-se quase integralmente nas opções OTM de Curta maturidade (MAPE de 126.75%). Contudo, o Erro Absoluto Médio (MAE) desse mesmo segmento foi de apenas R\$ 0,21. Como essas opções são negociadas por ínfimos centavos (próximas a perderem a validade sem exercício), uma predição desregulada em poucos centavos impulsiona o MAPE estruturalmente, mas resulta em um erro monetário inofensivo à operação financeira. Essa constatação valida a eficácia prática do algoritmo em todas as zonas de apreamento.

4.3 Análise de Sensibilidade e Coerência Econômica

Uma crítica comum à aplicação de métodos de *Machine Learning* em finanças é a opacidade (*black-box*) e o risco de o algoritmo aprender relações espúrias. Para verificar a consistência econômica da rede, procedeu-se a uma análise de sensibilidade empírica isolando o preço do ativo subjacente (S). Fixaram-se todas as outras variáveis de entrada (K, T, σ_{hist}, r) em suas medianas amostrais, variando-se apenas S em uma faixa de 50% a 150% do valor do *Strike*. A Figura 2 ilustra o comportamento preditivo da RNA frente a esse choque.

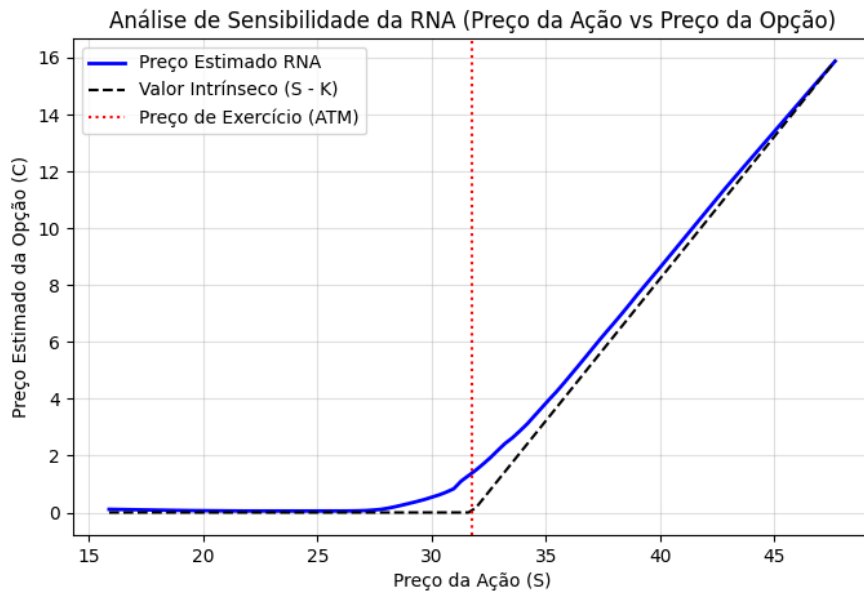


Figura 2: Análise de Sensibilidade da RNA (Preço da Ação vs Preço da Opção).

O gráfico demonstra que a Rede Neural internalizou as leis fundamentais da precificação de derivativos. Para preços do ativo substancialmente abaixo do *strike* (região OTM), a RNA prevê prêmios assintóticos próximos a zero, respeitando o limite inferior

do derivativo.

Na zona de transição (ATM), o modelo descreve uma curvatura suave, capturando o valor extrínseco gerado pela incerteza temporal. Por fim, à medida que a opção aprofunda no estado ITM, a derivada da curva preditiva converge para 1 (*Delta* unitário) e caminha de forma estritamente paralela à reta do valor intrínseco teórico ($\max(S - K, 0)$). Este comportamento atesta que a rede possui coerência e fundamentos de ausência de arbitragem propostos pela teoria financeira clássica.

4.4 Análise Gráfica e Sensibilidade por *Moneyness*

Com base na classificação de *moneyness* (ITM, ATM e OTM) a figura 3 ilustra a dispersão entre o preço real da opção e o valor predito pela RNA. O alinhamento acentuado dos pontos sobre a reta de identidade (Linha Ideal) corrobora a alta capacidade de generalização do algoritmo. Nota-se que as opções *In-The-Money* (ITM), que possuem prêmios intrínsecos mais elevados (localizadas na porção superior direita do gráfico), são precificadas com extrema precisão. Em contraste, observa-se uma maior dispersão relativa, embora com magnitudes absolutas baixas, nas opções *Out-of-The-Money* (OTM) e *At-The-Money* (ATM), concentradas próximas à origem dos eixos.

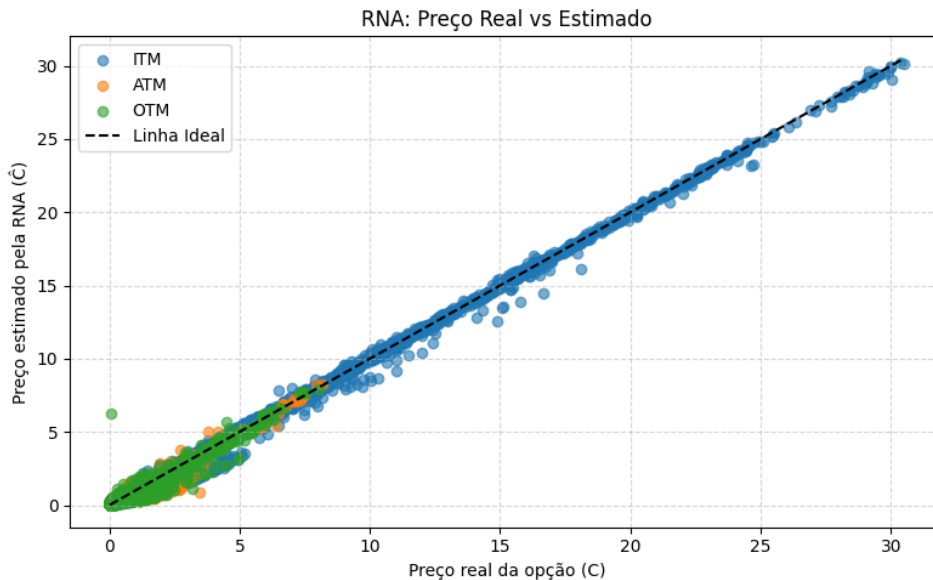


Figura 3: Dispersão entre o Preço Real e Estimado pela RNA classificado por *Moneyness*.

A relação estrutural entre a magnitude do erro e as características contratuais das opções é detalhada nas Figuras 4 e 5.

Ao examinar a Figura 4, constata-se que a variabilidade do erro de apreçamento

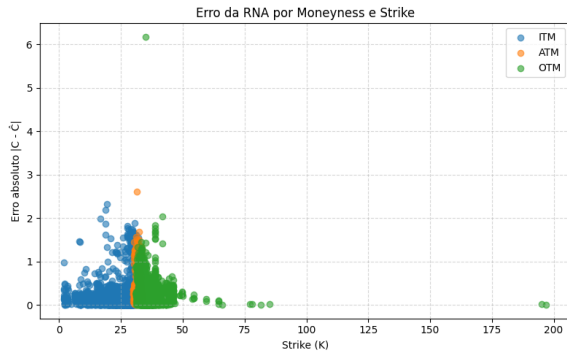


Figura 4: Erro Absoluto da RNA em função do Preço de Exercício (Strike).

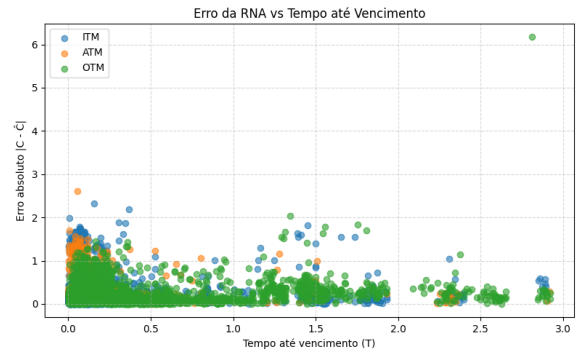


Figura 5: Erro Absoluto da RNA em função do Tempo até o Vencimento.

atinge seu ápice próximo à região em que as opções transitam do estado ITM para OTM (na vizinhança do dinheiro, ou ATM). Observam-se *outliers* pontuais de erro absoluto superior a R\$ 2,00, majoritariamente em opções ITM com *strikes* mais baixos. Opções profundamente OTM (com *strikes* superiores a R\$ 50,00) exibem erros residuais mínimos, o que é esperado dado que o valor intrínseco destas opções converge para zero.

A Figura 5 reforça a robustez temporal do modelo. A distribuição dos erros absolutos em relação ao tempo restante para o vencimento (T) revela uma dispersão heterocedástica leve, onde a maior concentração de desvios e *outliers* (como o ponto isolado de erro próximo a R\$ 6,00) ocorre em janelas temporais intermediárias a curtas. Apesar dessas observações atípicas, a nuvem de pontos densamente concentrada abaixo de R\$ 1,00 atesta a estabilidade do modelo em diferentes horizontes de maturação.

4.5 Importância das Variáveis (*Permutation Importance*)

Para interpretar o mecanismo de decisão adotado pela RNA, executou-se o algoritmo de *Permutation Importance*. Este método afere o grau de relevância de cada atributo de entrada mensurando o incremento relativo no MSE quando a referida variável é embaralhada aleatoriamente. Os resultados são expostos na Figura 6.

O gráfico evidencia a dependência estrutural do modelo em relação ao Preço de Exercício (*Strike* - K), cuja perturbação induziu um aumento assombroso de 400 vezes na função de erro. Esta magnitude atesta que a rede ancorou fortemente seu aprendizado na distância entre o preço *spot* e o exercício para definir o prêmio.

O Tempo até o vencimento (T) configurou-se como o segundo componente mais crítico, provocando um salto de cerca de 20 vezes no MSE, evidenciando a capacidade da rede em compreender a depreciação temporal inerente aos derivativos (*Time Decay*).

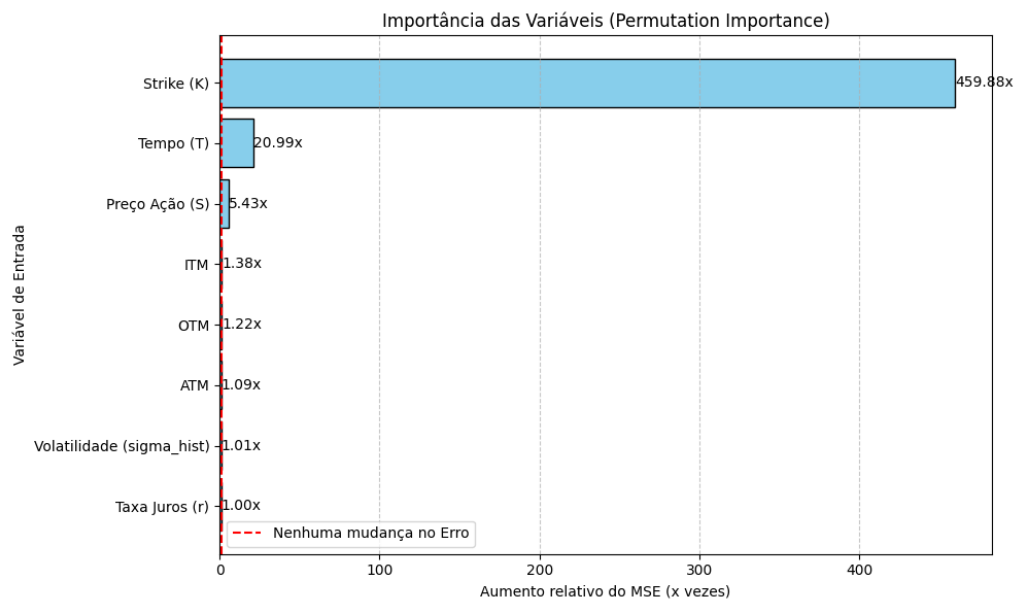


Figura 6: Aumento relativo do MSE via *Permutation Importance*.

Por outro lado, variáveis como o Preço da Ação (S) e as indicações binárias de *moneyness* exerceram um impacto moderado (entre 2,87 e 4,78 vezes o erro basal).

5 Conclusão

O presente relatório cumpriu com êxito o objetivo de desenvolver, treinar e avaliar uma Rede Neural Artificial (RNA) capaz de precificar opções de compra europeias do ativo PETR4. O estudo contrastou o desempenho da inteligência artificial não apenas com o tradicional modelo analítico de Black-Scholes (BS), mas também com abordagens estatísticas paramétricas tradicionais, como a Regressão Linear Múltipla.

A análise exploratória e estatística validou a premissa que motivou a adoção de métodos de *Deep Learning* neste estudo. Os testes de Shapiro-Wilk e Jarque-Bera comprovaram a quebra da hipótese de normalidade dos retornos logarítmicos do ativo subjacente. Essa não conformidade com o Movimento Browniano Geométrico (MBG) afeta intrinsecamente a precisão teórica de Black-Scholes, justificando a utilização de um aproximador universal não-paramétrico capaz de modelar a assimetria e a curtose presentes nos dados reais do mercado.

Em termos de acurácia preditiva, a RNA demonstrou superioridade, obteve um Erro Quadrático Médio (MSE) de 0,1161, superando a formulação clássica de Black-Scholes (0,2708) e evidenciando a natureza não-linear do prêmio das opções demonstrar desempenho superior a Regressão Linear (MSE de 5,96). A avaliação estratificada por *moneyness* e maturidade elucidou que a rede opera com maior precisão em opções *In-The-Money* (ITM) longas — atingindo um Erro Percentual Absoluto Médio (MAPE) de 2,78% —, além de demonstrar robustez nas zonas *Out-of-The-Money* (OTM) de curto prazo, onde o erro financeiro absoluto manteve-se baixo.

Adicionalmente, a pesquisa mitigou o problema de *black-box* inerente às RNAs. A análise empírica de sensibilidade comprovou que a rede internalizou as leis da economia, replicando a curva convexa do prêmio e respeitando os limites teóricos de ausência de arbitragem. Paralelamente, a análise via *Permutation Importance* revelou que o algoritmo concentrou seu aprendizado no Preço de Exercício (K) e no Tempo até o vencimento (T), extraindo a dinâmica do prêmio de risco diretamente da estrutura do derivativo.

Referências

- CYBENKO, G. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, Springer, v. 2, n. 4, p. 303–314, 1989.
- HAYKIN, S. *Neural Networks and Learning Machines*. 3rd. ed.. ed. Upper Saddle River, NJ: Prentice Hall, 2009. ISBN 9780131471399.
- HORNIK, K. Approximation capabilities of multilayer feedforward networks. *Neural networks*, Elsevier, v. 4, n. 2, p. 251–257, 1991.
- HULL, J. C. *Options, Futures, and Other Derivatives*. 10. ed.. ed. Boston, MA: Pearson, 2018.
- HUTCHINSON, J. M.; LO, A. W.; POGGIO, T. A nonparametric approach to pricing and hedging derivative securities via learning networks. *The Journal of Finance*, Wiley, v. 49, n. 3, p. 1–51, 1994.
- MCCULLOCH, W. S.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, Springer, v. 5, n. 4, p. 115–133, 1943.
- RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, Nature Publishing Group, v. 323, n. 6088, p. 533–536, 1986.
- TORRES, R. et al. Model-free local recalibration of neural networks. 2024. Disponível em: <https://arxiv.org/abs/2403.05756>.

Apêndice

A Leitura do histórico diário de negociações dos ativos da bolsa

```
1 import pandas as pd
2
3 arquivo = "COTAHIST_A2025.TXT"
4 # layout oficial B3 COTAHIST
5
6 colspecs = [
7     (0, 2),
8     (2, 10),
9     (10, 12),
10    (12, 24),
11    (24, 27),
12    (27, 39),
13    (39, 49),
14    (49, 52),
15    (52, 56),
16    (56, 69),
17    (69, 82),
18    (82, 95),
19    (95, 108),
20    (108, 121),
21    (121, 134),
22    (134, 147),
23    (147, 152),
24    (152, 170),
25    (170, 188),
26    (188, 201),
27    (201, 202),
28    (202, 210),
29    (210, 217),
30    (217, 230),
31    (230, 242),
32    (242, 245),
33 ]
34
35 cols = [
36     "tipo_registro",
37     "data",
38     "cod_bdi",
39     "ticker",
```

```

40     "tipo_mercado",
41     "nome_empresa",
42     "especificacao",
43     "prazo_dias",
44     "moeda",
45     "preco_abertura",
46     "preco_max",
47     "preco_min",
48     "preco_medio",
49     "preco_ultimo",
50     "preco_melhor_compra",
51     "preco_melhor_venda",
52     "num_negocios",
53     "quantidade",
54     "volume",
55     "preco_exercicio",
56     "indicador_correcao",
57     "data_vencimento",
58     "fator_cotacao",
59     "preco_exercicio_pontos",
60     "cod_isin",
61     "distribuicao"
62 ]
63
64 df = pd.read_fwf(
65     arquivo,
66     colspecs=colspecs,
67     names=cols,
68     encoding="latin1"
69 )
70
71 df = df[df["tipo_registro"] == 1]
72
73 df["data"] = pd.to_datetime(df["data"], format="%Y%m%d")
74 df["data_vencimento"] = pd.to_datetime(df["data_vencimento"], format="%Y
    %m%d", errors="coerce")
75
76 for c in ["preco_ultimo", "preco_abertura", "preco_max", "preco_min", "
    preco_exercicio"]:
77     df[c] = df[c] / 100
78
79 df.to_parquet("b3_2025.parquet")

```

B Seleção das opções de compra da Petrobrás

```

1 import pandas as pd

```

```
2
3 df = pd.read_parquet("b3_2025.parquet")
4
5 # manter s    registros v lidos
6 df = df[df["tipo_registro"] == 1]
7 # tipo mercado 70 = op    es
8 opcoes = df[df["tipo_mercado"] == 70].copy()
9
10 # apenas Petrobras
11 petr_opcoes = opcoes[opcoes["ticker"].str.startswith("PETR")].copy()
12
13 petr_opcoes["serie"] = petr_opcoes["ticker"].str[4]
14
15 calls_letters = list("ABCDEFGHijkl")
16
17 petr_opcoes["tipo"] = petr_opcoes["serie"].apply(
18     lambda x: "CALL" if x in calls_letters else "PUT"
19 )
20 petr_opcoes["C"] = petr_opcoes["preco_ultimo"]
21 petr_opcoes["K"] = petr_opcoes["preco_exercicio"]
22
23 spot = df[df["ticker"] == "PETR4"][["data", "preco_ultimo"]].copy()
24 spot.rename(columns={"preco_ultimo": "S"}, inplace=True)
25
26 petr_opcoes = petr_opcoes.merge(spot, on="data", how="left")
27
28 petr_opcoes["T"] = (
29     (petr_opcoes["data_vencimento"] - petr_opcoes["data"])
30     .dt.days / 252
31 )
32
33 petr_opcoes = petr_opcoes[petr_opcoes["T"] > 0]
34
35 petr_opcoes = petr_opcoes[
36     (petr_opcoes["C"] > 0.01) &
37     (petr_opcoes["S"] > 1) &
38     (petr_opcoes["K"] > 1)
39 ]
40
41 petr_opcoes[["ticker", "data", "tipo", "S", "K", "T", "C"]].to_csv(
42     "base_black_scholes_petr4_2025.csv",
43     index=False
44 )
```

C Normalidade de BS

O código abaixo contém o teste de normalidade sobre os ativos subjacentes da Petrobrás:

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import scipy.stats as stats
5 import seaborn as sns
6 from scipy.stats import skew, kurtosis, jarque_bera
7 # 1. Carregue seus dados históricos da ATO (nó das operações)
8 # Supondo que você tenha um csv com datas e preços de fechamento
9 # df_acao = pd.read_csv('historico_petr4.csv')
10 # Ou baixe agora se não tiver:
11 import yfinance as yf
12 df_acao = yf.download('PETR4.SA', start='2025-01-01', end='2025-12-31')[
    'Close']
13
14 # 2. Calcular os Retornos Logarítmicos (Conforme Eq 3.2.1 do seu PDF)
15 #  $u_i = \ln(S_i / S_{i-1})$ 
16 log_returns = np.log(df_acao / df_acao.shift(1)).dropna()
17
18 # 3. Visualiza o Gráfico (Histograma vs Curva Normal)
19 plt.figure(figsize=(10, 6))
20 sns.histplot(log_returns, kde=True, stat="density", label="Dados Reais",
    color="blue")
21
22 # Gerar curva normal teórica com mesma média e desvio padrão
23 mu, std = stats.norm.fit(log_returns)
24 xmin, xmax = plt.xlim()
25 x = np.linspace(xmin, xmax, 100)
26 p = stats.norm.pdf(x, mu, std)
27 plt.plot(x, p, 'r', linewidth=2, label="Distribuição Normal (Teórica)"
    ")
28
29 plt.title("Distribuição dos Retornos Logarítmicos vs Normal Teórica"
    )
30 plt.legend()
31 plt.show()
32
33
34 # 5. Testes Estatísticos Formais
35 shapiro_stat, shapiro_p = stats.shapiro(log_returns)
36 jarque_bera_stat, jarque_bera_p = stats.jarque_bera(log_returns)
37

```

```
38
39 print(f"--- Resultados dos Testes de Normalidade ---")
40 print(f"Shapiro-Wilk: p-valor = {shapiro_p:.5f} (H0:    Normal)")
41 print(f"Jarque-Bera: p-valor = {jarque_bera_p:.5f} (H0:    Normal)")
42
43
44 if shapiro_p < 0.05:
45     print("\nCONCLUSÃO: Rejeitamos a hipótese nula. Os dados NÃO
46     seguem uma distribuição Normal.")
47     print("Isso justifica o uso de Redes Neurais, que não exigem essa
48     premissa!")
49 else:
50     print("--- Resultados dos Testes de Normalidade ---")
```

D Código Fonte em Python: Precificação de Opções com RNA

O script a seguir contém toda a rotina computacional desenvolvida para este trabalho, incluindo o tratamento da base de dados, a implementação da Rede Neural Artificial via *PyTorch*, o cálculo das métricas de avaliação e a geração das análises gráficas. O modelo foi configurado com uma semente de aleatoriedade (*seed*) fixa para garantir a reprodutibilidade dos resultados.

```
1 import os
2 import random
3 import numpy as np
4 import pandas as pd
5 import yfinance as yf
6
7 import torch
8 import torch.nn as nn
9 import torch.optim as optim
10 from torch.utils.data import DataLoader, TensorDataset
11
12 from sklearn.model_selection import train_test_split
13 from sklearn.preprocessing import StandardScaler
14 from sklearn.metrics import mean_squared_error, mean_absolute_error,
15     mean_absolute_percentage_error
16 from sklearn.linear_model import LinearRegression
17
18 from scipy.stats import norm
19 import matplotlib.pyplot as plt
20 import copy
```

```

21 from volatilidade_historica import calcular_volatilidade_historica
22
23 def set_seed(seed=42):
24     os.environ['PYTHONHASHSEED'] = str(seed)
25     random.seed(seed)
26     np.random.seed(seed)
27     torch.manual_seed(seed)
28     if torch.cuda.is_available():
29         torch.cuda.manual_seed(seed)
30         torch.cuda.manual_seed_all(seed)
31     torch.backends.cudnn.deterministic = True
32     torch.backends.cudnn.benchmark = False
33
34 set_seed(42)
35
36 df = pd.read_csv("base_black_scholes_petr4_2025.csv")
37 df = df[df["tipo"] == "CALL"].reset_index(drop=True)
38 df['data'] = pd.to_datetime(df['data'])
39 df = df.sort_values(by='data').reset_index(drop=True)
40
41 petr4 = yf.download("PETR4.SA", start="2024-11-01", end="2025-12-31",
42                     progress=False)['Close']
43 petr4 = petr4.reset_index()
44
45 if isinstance(petr4.columns, pd.MultiIndex):
46     petr4.columns = petr4.columns.get_level_values(0)
47
48 petr4.columns = ['data', 'S_hist']
49 petr4['data'] = pd.to_datetime(petr4['data']).dt.tz_localize(None)
50 petr4['sigma_hist'] = calcular_volatilidade_historica(petr4['S_hist'],
51                                                       janelas=20)
52
53 df = pd.merge(df, petr4[['data', 'sigma_hist']], on='data', how='left')
54 df['sigma_hist'] = df['sigma_hist'].ffill()
55
56 def classify_moneyness(row, eps=0.02):
57     if abs(row["S"] - row["K"]) / row["K"] <= eps:
58         return "ATM"
59     elif row["S"] > row["K"]:
60         return "ITM"
61     else:
62         return "OTM"
63
64 df["moneyness"] = df.apply(classify_moneyness, axis=1)
65 df = pd.get_dummies(df, columns=["moneyness"])

```



```

66 df['r'] = 0.135
67 colunas_cont = ["S", "K", "T", "sigma_hist", "r"]
68 X_cont = df[colunas_cont].values
69 colunas_cat = ["moneyness_ITM", "moneyness_ATM", "moneyness_OTM"]
70 X_cat = df[colunas_cat].astype(float).values
71 y = df["C"].values.reshape(-1, 1)
72
73 indices = np.arange(len(df))
74 idx_train, idx_temp = train_test_split(indices, test_size=0.2, shuffle=
    False)
75 idx_val, idx_test = train_test_split(idx_temp, test_size=0.5, shuffle=
    False)
76
77 df_test = df.iloc[idx_test].copy()
78
79 X_cont_train, X_cont_val, X_cont_test = X_cont[idx_train], X_cont[
    idx_val], X_cont[idx_test]
80 X_cat_train, X_cat_val, X_cat_test = X_cat[idx_train], X_cat[idx_val],
    X_cat[idx_test]
81 y_train_raw, y_val_raw, y_test_raw = y[idx_train], y[idx_val], y[
    idx_test]
82
83 scaler_X = StandardScaler()
84 X_cont_train_scaled = scaler_X.fit_transform(X_cont_train)
85 X_cont_val_scaled = scaler_X.transform(X_cont_val)
86 X_cont_test_scaled = scaler_X.transform(X_cont_test)
87
88 scaler_y = StandardScaler()
89 y_train_scaled = scaler_y.fit_transform(y_train_raw)
90 y_val_scaled = scaler_y.transform(y_val_raw)
91 y_test_scaled = scaler_y.transform(y_test_raw)
92
93 X_train_final = np.hstack([X_cont_train_scaled, X_cat_train])
94 X_val_final = np.hstack([X_cont_val_scaled, X_cat_val])
95 X_test_final = np.hstack([X_cont_test_scaled, X_cat_test])
96
97 X_train_tensor = torch.tensor(X_train_final, dtype=torch.float32)
98 y_train_tensor = torch.tensor(y_train_scaled, dtype=torch.float32)
99 X_val_tensor = torch.tensor(X_val_final, dtype=torch.float32)
100 y_val_tensor = torch.tensor(y_val_scaled, dtype=torch.float32)
101 X_test_tensor = torch.tensor(X_test_final, dtype=torch.float32)
102 y_test_tensor = torch.tensor(y_test_scaled, dtype=torch.float32)
103
104 train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
105 train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
106
107 class OptionPricingNN(nn.Module):

```

```
108     def __init__(self):
109         super().__init__()
110         self.net = nn.Sequential(
111             nn.Linear(8, 128),
112             nn.ReLU(),
113             nn.Linear(128, 64),
114             nn.ReLU(),
115             nn.Linear(64, 32),
116             nn.ReLU(),
117             nn.Linear(32, 1)
118         )
119
120     def forward(self, x):
121         return self.net(x)
122
123 model = OptionPricingNN()
124 criterion = nn.MSELoss()
125 optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
126
127 epochs = 200
128 patience = 30
129 best_val_loss = float('inf')
130 epochs_no_improve = 0
131 best_model_wts = copy.deepcopy(model.state_dict())
132
133 print("\nIniciando treinamento da RNA...")
134 for epoch in range(epochs):
135     model.train()
136     train_loss = 0.0
137     for batch_X, batch_y in train_loader:
138         optimizer.zero_grad()
139         y_pred = model(batch_X)
140         loss = criterion(y_pred, batch_y)
141         loss.backward()
142         optimizer.step()
143         train_loss += loss.item() * batch_X.size(0)
144
145     train_loss /= len(train_loader.dataset)
146
147     model.eval()
148     with torch.no_grad():
149         val_pred = model(X_val_tensor)
150         val_loss = criterion(val_pred, y_val_tensor).item()
151
152     if val_loss < best_val_loss:
153         best_val_loss = val_loss
154         best_model_wts = copy.deepcopy(model.state_dict())
```

```

155         epochs_no_improve = 0
156     else:
157         epochs_no_improve += 1
158
159     if epoch % 50 == 0 or epochs_no_improve == patience:
160         print(f"Epoch {epoch:3d} | Train MSE: {train_loss:.4f} | Val MSE
161 : {val_loss:.4f}")
162
163     if epochs_no_improve == patience:
164         print(f"Early stopping acionado na poca {epoch}!")
165         break
166
167 model.load_state_dict(best_model_wts)
168 torch.save(model.state_dict(), "melhor_modelo_rna_petr4.pth")
169 print("\nModelo salvo com sucesso no arquivo: 'melhor_modelo_rna_petr4.
170 pth'!")
171
172 def safe_mape(y_true, y_pred):
173     return mean_absolute_percentage_error(y_true + 1e-8, y_pred) * 100
174
175 model.eval()
176 with torch.no_grad():
177     test_pred_scaled = model(X_test_tensor)
178 test_pred_rna = scaler_y.inverse_transform(test_pred_scaled.numpy()).
179     flatten()
180
181 lr_model = LinearRegression()
182 lr_model.fit(X_train_final, y_train_raw.flatten())
183 test_pred_lr = lr_model.predict(X_test_final)
184
185 def black_scholes_call(S, K, T, r, sigma):
186     d1 = (np.log(S / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * np.sqrt
187 (T))
188     d2 = d1 - sigma * np.sqrt(T)
189     return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
190
191 df_test["C_BS"] = black_scholes_call(
192     df_test["S"].values, df_test["K"].values, df_test["T"].values,
193     df_test["r"].values, df_test["sigma_hist"].values
194 )
195 test_pred_bs = df_test["C_BS"].values
196 y_true = df_test["C"].values
197
198 print("\n=== COMPARA O DE MODELOS (CONJUNTO DE TESTE) ===")
199 for name, preds in [("Rede Neural", test_pred_rna), ("Reg. Linear
200 M ltipla", test_pred_lr),
201 ("Black-Scholes", test_pred_bs)]:

```

```

197     mse = mean_squared_error(y_true, preds)
198     mae = mean_absolute_error(y_true, preds)
199     mape = safe_mape(y_true, preds)
200     print(f"[{name}] MSE: {mse:.4f} | MAE: R$ {mae:.4f} | MAPE: {mape:.2f}%")
201
202 df_test["C_RNA"] = test_pred_rna
203
204 def calc_metrics(df_subset, true_col, pred_col):
205     if len(df_subset) == 0: return np.nan, np.nan, np.nan
206     y_t = df_subset[true_col].values
207     y_p = df_subset[pred_col].values
208     return mean_squared_error(y_t, y_p), mean_absolute_error(y_t, y_p),
209         safe_mape(y_t, y_p)
210
211 df_test['maturidade'] = pd.cut(df_test['T'], bins=[-np.inf, 0.25, 0.75,
212     np.inf], labels=['Curta', 'M dia', 'Longa'])
213
214 print("\n DESEMPENHO DA RNA POR MONEYNNESS E MATURIDADE ")
215 for moneyness in ["ITM", "ATM", "OTM"]:
216     for mat in ["Curta", "M dia", "Longa"]:
217         subset = df_test[(df_test[f"moneyness_{moneyness}"] == 1) & (
218             df_test['maturidade'] == mat)]
219         mse, mae, mape = calc_metrics(subset, "C", "C_RNA")
220         if not np.isnan(mse):
221             print(
222                 f"[{moneyness} | {mat:<5}] N={len(subset):<4} | MSE: {mse:.4f} | MAE: R$ {mae:.4f} | MAPE: {mape:.2f}%")
223
224 print("\n--- CALCULANDO A AN LISE DE SENSIBILIDADE (DELTA EMP RICO)
225     ---")
226
227 median_K = np.median(df_test["K"])
228 median_T = np.median(df_test["T"])
229 median_sigma = np.median(df_test["sigma_hist"])
230 median_r = 0.135
231
232 S_range = np.linspace(median_K * 0.5, median_K * 1.5, 100)
233
234 sens_df = pd.DataFrame({
235     "S": S_range,
236     "K": median_K,
237     "T": median_T,
238     "sigma_hist": median_sigma,
239     "r": median_r
240 })

```

```

238 sens_df["moneyness"] = sens_df.apply(classify_moneyness, axis=1)
239 sens_df = pd.get_dummies(sens_df, columns=["moneyness"])
240 for col in ["moneyness_ITM", "moneyness_ATM", "moneyness_OTM"]:
241     if col not in sens_df: sens_df[col] = 0
242
243 X_cont_sens = scaler_X.transform(sens_df[["S", "K", "T", "sigma_hist", "
    r"]].values)
244 X_cat_sens = sens_df[["moneyness_ITM", "moneyness_ATM", "moneyness_OTM"
    ]].astype(float).values
245 X_sens_final = np.hstack([X_cont_sens, X_cat_sens])
246 X_sens_tensor = torch.tensor(X_sens_final, dtype=torch.float32)
247
248 model.eval()
249 with torch.no_grad():
250     C_pred_scaled = model(X_sens_tensor)
251
252 C_pred_sens = scaler_y.inverse_transform(C_pred_scaled.numpy()).flatten
    ()
253 C_teorico = np.maximum(S_range - median_K, 0)
254
255 plt.figure(figsize=(8, 5))
256 plt.plot(S_range, C_pred_sens, label="Pre o Estimado RNA", color="blue"
    , linewidth=2)
257 plt.plot(S_range, C_teorico, label="Valor Intr nseco (S - K)", color="
    black", linestyle="--")
258 plt.axvline(x=median_K, color='red', linestyle=':', label="Pre o de
    Exerc cio (ATM)")
259 plt.xlabel("Pre o da A o (S)")
260 plt.ylabel("Pre o Estimado da Op o (C)")
261 plt.title("An lise de Sensibilidade da RNA (Pre o da A o vs Pre o
    da Op o)")
262 plt.legend()
263 plt.grid(alpha=0.4)
264 plt.show()
265
266 df_test["C_RNA"] = test_pred_rna
267 df_test["erro_abs"] = np.abs(df_test["C"] - df_test["C_RNA"])
268
269 plt.figure(figsize=(8, 5))
270 for label in ["ITM", "ATM", "OTM"]:
271     subset = df_test[df_test[f"moneyness_{label}"] == 1]
272     plt.scatter(subset["K"], subset["erro_abs"], label=label, alpha=0.6)
273 plt.xlabel("Strike (K)")
274 plt.ylabel("Erro absoluto |C - |")
275 plt.title("Erro da RNA por Moneyness e Strike")
276 plt.legend()
277 plt.grid(True, linestyle='--', alpha=0.5)

```

```

278 plt.tight_layout()
279 plt.show()
280
281 plt.figure(figsize=(8, 5))
282 for label in ["ITM", "ATM", "OTM"]:
283     subset = df_test[df_test[f"moneyiness_{label}"] == 1]
284     plt.scatter(subset["C"], subset["C_RNA"], label=label, alpha=0.6)
285 min_c, max_c = df_test["C"].min(), df_test["C"].max()
286 plt.plot([min_c, max_c], [min_c, max_c], color='black', linestyle='--',
287         label="Linha Ideal")
288 plt.xlabel("Pre o real da op o (C)")
289 plt.ylabel("Pre o estimado pela RNA ( )")
290 plt.title("RNA: Pre o Real vs Estimado")
291 plt.legend()
292 plt.grid(True, linestyle='--', alpha=0.5)
293 plt.tight_layout()
294 plt.show()
295
296 plt.figure(figsize=(8, 5))
297 for label in ["ITM", "ATM", "OTM"]:
298     subset = df_test[df_test[f"moneyiness_{label}"] == 1]
299     plt.scatter(subset["T"], subset["erro_abs"], label=label, alpha=0.6)
300 plt.xlabel("Tempo at vencimento (T)")
301 plt.ylabel("Erro absoluto |C - |")
302 plt.title("Erro da RNA vs Tempo at Vencimento")
303 plt.legend()
304 plt.grid(True, linestyle='--', alpha=0.5)
305 plt.tight_layout()
306 plt.show()
307
308 feature_names = [
309     "Pre o A o (S)",
310     "Strike (K)",
311     "Tempo (T)",
312     "Volatilidade (sigma_hist)",
313     "Taxa Juros (r)",
314     "ITM",
315     "ATM",
316     "OTM"
317 ]
318
319 def calculate_permutation_importance(model, X_tensor, y_tensor,
320 criterion, feature_names):
321     model.eval()
322
323     with torch.no_grad():
324         base_pred = model(X_tensor)

```

```
323     base_loss = criterion(base_pred, y_tensor).item()
324
325     importances = []
326
327     for i in range(X_tensor.shape[1]):
328         X_permuted = X_tensor.clone()
329
330         indices_embaralhados = torch.randperm(X_tensor.shape[0])
331         X_permuted[:, i] = X_tensor[indices_embaralhados, i]
332
333         with torch.no_grad():
334             permuted_pred = model(X_permuted)
335             permuted_loss = criterion(permuted_pred, y_tensor).item()
336
337             aumento_erro = permuted_loss / base_loss
338             importances.append(aumento_erro)
339
340     return importances
341
342 importances = calculate_permutation_importance(model, X_test_tensor,
343                                               y_test_tensor, criterion, feature_names)
344
345 importance_df = pd.DataFrame({
346     'Variavel': feature_names,
347     'Aumento_MSE': importances
348 }).sort_values(by='Aumento_MSE', ascending=True)
349
350 plt.figure(figsize=(10, 6))
351 bars = plt.barh(importance_df['Variavel'], importance_df['Aumento_MSE'],
352                 color='skyblue', edgecolor='black')
353 plt.axvline(x=1.0, color='red', linestyle='--', label='Nenhuma mudan a
354             no Erro')
355 plt.xlabel("Aumento relativo do MSE (x vezes)")
356 plt.ylabel("Vari vel de Entrada")
357 plt.title("Import ncia das Vari veis (Permutation Importance)")
358 plt.legend()
359 plt.grid(axis='x', linestyle='--', alpha=0.7)
360
361 for bar in bars:
362     plt.text(
363         bar.get_width() + 0.05,
364         bar.get_y() + bar.get_height() / 2,
365         f"{bar.get_width():.2f}x",
366         va='center'
367     )
368
369 plt.tight_layout()
```

```
367 plt.show()
```

E nome apêndice 2

texto...

Anexo

A nome anexo 1

texto...